

Interpolation

Exemples et motivations

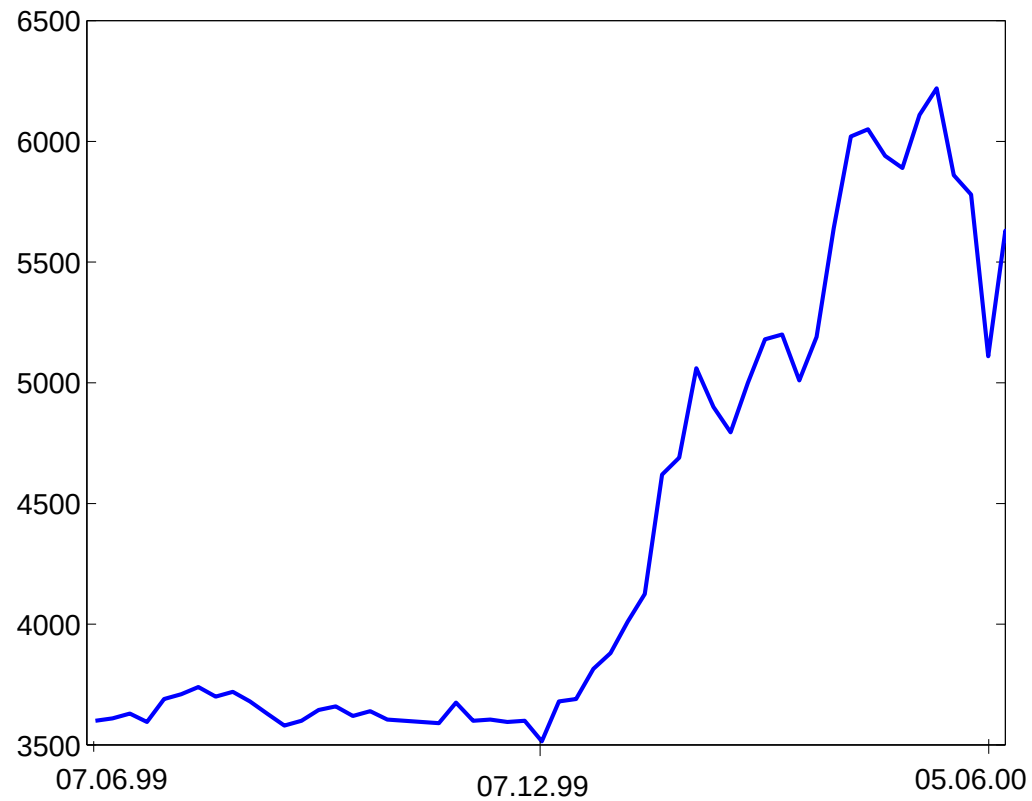
Exemple 1 Il est bien connu depuis plus d'un siècle que la température de l'air à proximité du sol varie selon la concentration de l'acide carbonique. En particulier, le tableau suivant (extrait de "Philosophical Magazine" 41, 237 (1896)) montre les variations annuelles de la température moyenne pour différentes latitudes en fonction de la concentration (relative) K de l'acide carbonique dans l'atmosphère (on suppose la concentration actuelle égale à 1).

Latitude	K=0.67	K=1.5	K=2
55	-3.228	3.621	6.055
45	-3.309	3.652	5.922
35	-3.327	3.522	5.707
25	-3.172	3.476	5.308
15	-3.074	3.253	5.020
5	-3.029	3.152	4.957

Latitude	K=0.67	K=1.5	K=2
-5	-3.029	3.150	4.974
-15	-3.124	3.207	5.078
-25	-3.209	3.274	5.355
-35	-3.350	3.529	5.627
-45	-3.373	3.705	5.954
-55	-3.258	3.704	6.103

Les techniques d'interpolation permettent de reconstruire, à partir des données disponibles, les valeurs de température pour des latitudes ou des concentrations non contenues dans le tableau. Par exemple, on aimerait savoir la variation de la température à la latitude de Lausanne (46.316).

Exemple 2 La figure qui suit montre la valeur d'une action à la bourse de Zurich dans la période 07/06/1999, 05/06/2000.



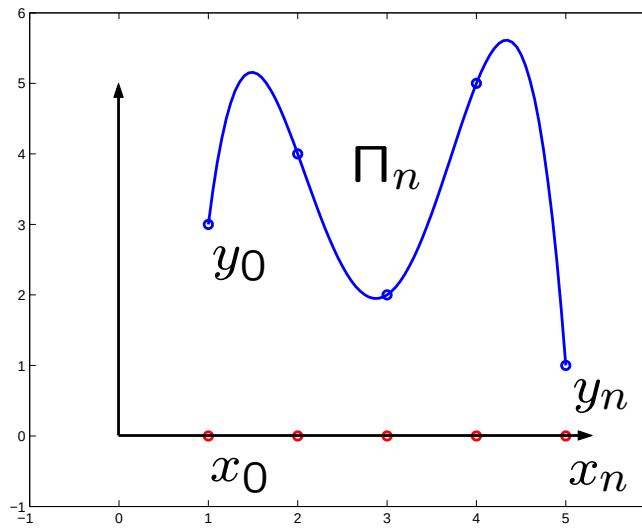
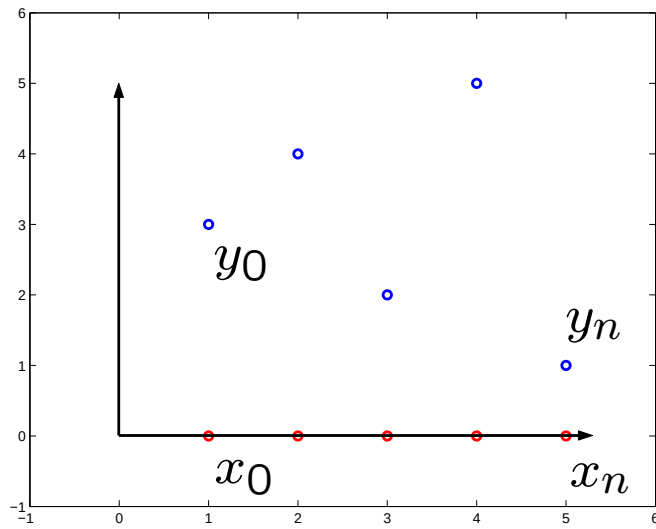
À partir de ces données, on aimerait “deviner” le comportement de l'action au début du mois de Juillet 2000.

Position du problème (Sec. 3.1 du livre)

Soit $n \geq 0$ un nombre entier. Étant donnés $n + 1$ points distincts $x_0, x_1, \dots, x_n$ et $n + 1$ valeurs $y_0, y_1, \dots, y_n$, on cherche un polynôme p de degré n , tel que

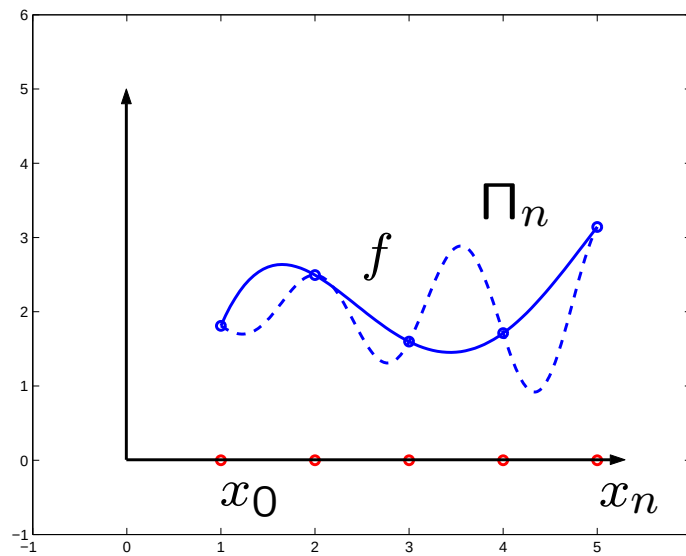
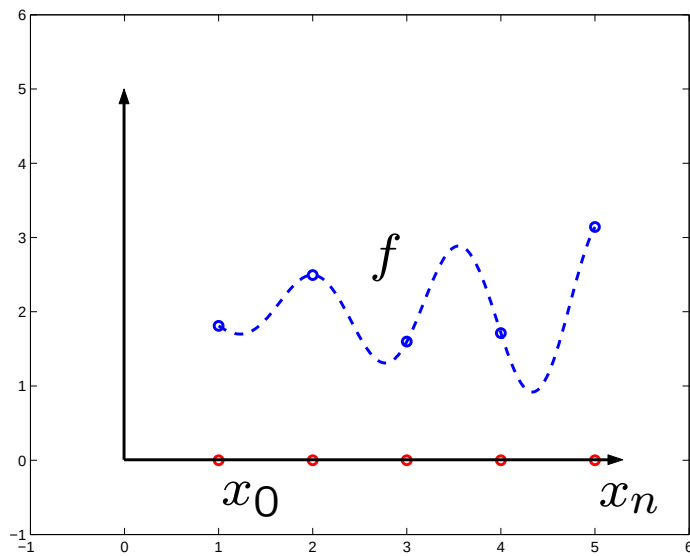
$$p(x_j) = y_j \quad \text{pour } 0 \leq j \leq n. \quad (1)$$

Dans le cas affirmatif, on note $p = \Pi_n$ et on appelle Π_n le polynôme d'interpolation aux points $x_j, j = 0, \dots, n$.



($n = 4$)

Soit $f \in C^0(I)$ et $x_0, \dots, x_n \in I$. Si comme valeurs y_j on prend $y_j = f(x_j)$, $0 \leq j \leq n$, alors le polynôme d'interpolation $\Pi_n(x)$ est noté $\Pi_n f(x)$ et est appelé l'interpolant de f aux points $x_0, \dots, x_n$.



$(n = 4)$

Base de Lagrange (Sec. 3.1.1 du livre)

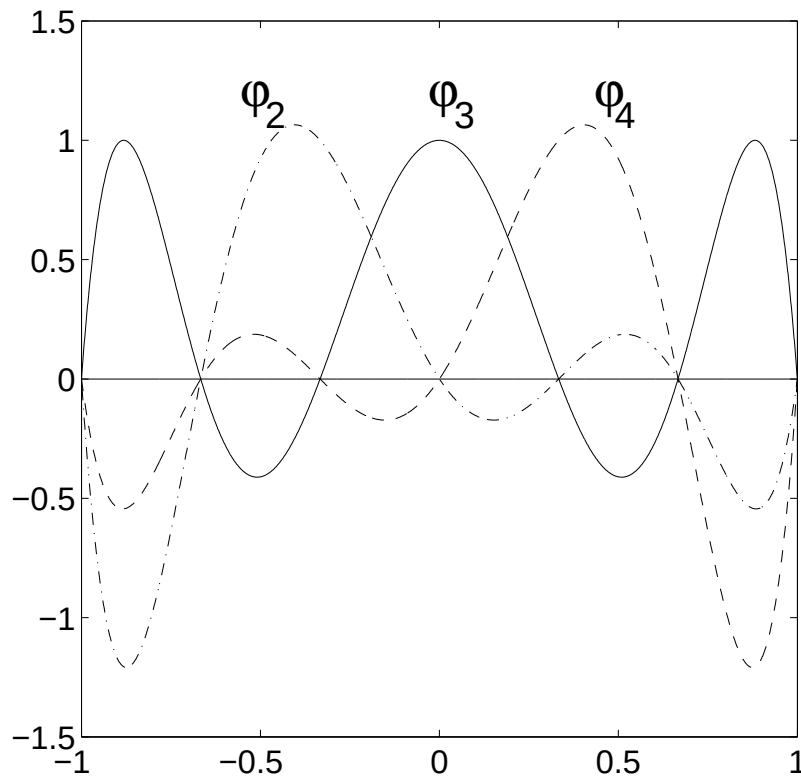
On considère les polynômes φ_k , $k = 0, \dots, n$ de degré n tels que

$$\varphi_k(x_j) = \delta_{jk}, \quad k, j = 0, \dots, n,$$

où $\delta_{jk} = 1$ si $j = k$ et $\delta_{jk} = 0$ si $j \neq k$. Explicitement, on a

$$\varphi_k(x) = \prod_{j=0, j \neq k}^n \frac{(x - x_j)}{(x_k - x_j)}.$$

La figure qui suit montre trois polynômes de Lagrange de degré 6 relatifs aux points d'interpolations $x_0 = -1$, $x_1 = -2/3, \dots, x_5 = 2/3$, et $x_6 = 1$.



$(n = 6)$

Exemple 3 Pour $n = 2$, $x_0 = -1$, $x_1 = 0$, $x_2 = 1$ les polynômes de la base de Lagrange sont

$$\begin{aligned}\varphi_0(x) &= \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} = \frac{1}{2}x(x - 1), \\ \varphi_1(x) &= \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} = -(x + 1)(x - 1), \\ \varphi_2(x) &= \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} = \frac{1}{2}x(x + 1).\end{aligned}$$

Le polynôme d'interpolation Π_n des valeurs y_j aux points x_j , $j = 0, \dots, n$, s'écrit

$$\Pi_n(x) = \sum_{k=0}^n y_k \varphi_k(x), \quad (2)$$

car il vérifie $\Pi_n(x_j) = \sum_{k=0}^n y_k \varphi_k(x_j) = y_j$.

Par conséquent on aura $\Pi_n f(x) = \sum_{k=0}^n f(x_k) \varphi_k(x)$.

On montre maintenant que le polynôme Π_n en (2) est le seul polynôme de degré n interpolant les données y_i aux nœuds x_i . Soit, en effet, $Q_n(x)$ un autre polynôme d'interpolation. Alors on a

$$Q_n(x_j) - \Pi_n(x_j) = 0, \quad j = 0, \dots, n.$$

Donc, $Q_n(x) - \Pi_n(x)$ est un polynôme de degré n qui s'annule en $n + 1$ points distincts; il en suit que $Q_n = \Pi_n$, d'où l'unicité du polynôme interpolant.

Interpolation d'une fonction continue

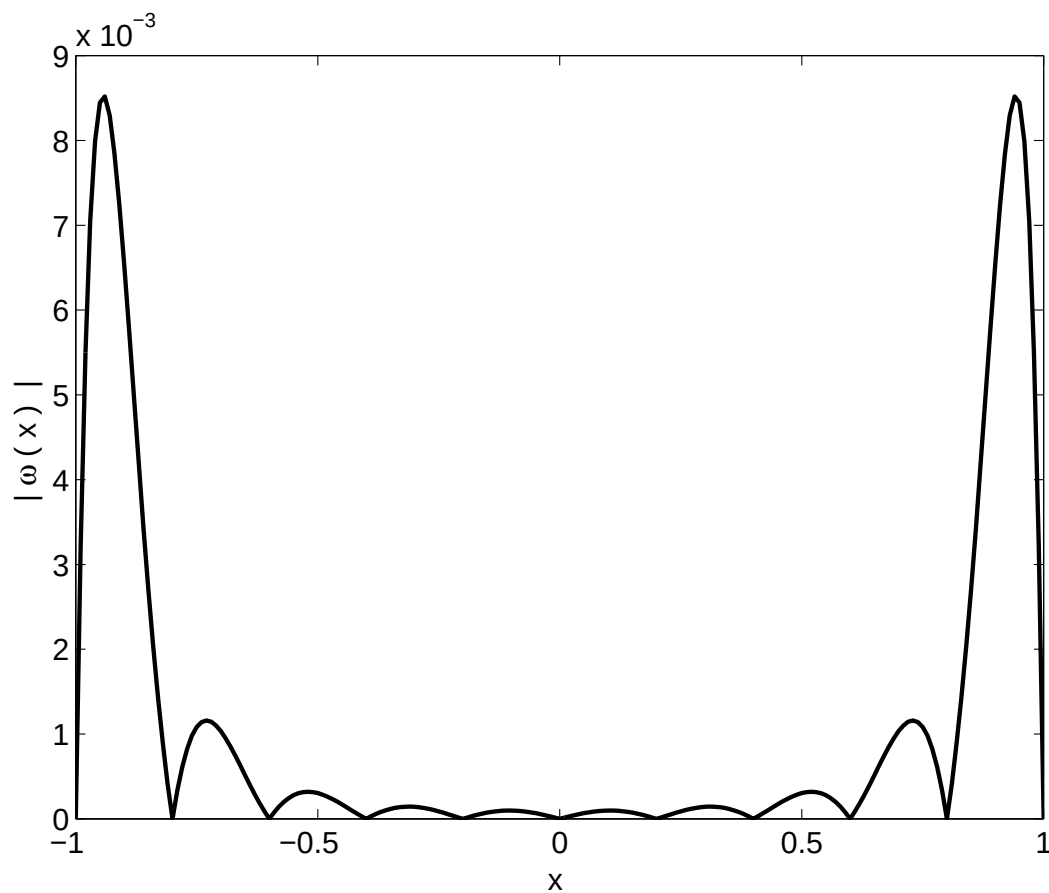
Théorème 1 (Erreur d'interpolation) (*Proposition 3.2 du livre*)

Soient $x_0, x_1, \dots, x_n$, $n + 1$ nœuds distincts dans $I = [a, b]$ et soit $f \in C^{n+1}(I)$. Alors, pour tout $x \in I$

$$E_n f(x) = f(x) - \Pi_n f(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x)$$

où $\omega_{n+1}(x) = \prod_{i=0}^n (x - x_i)$ et $\xi \in I$.

La figure ci-dessous montre la valeur absolue de $\omega_{n+1}(x)$ pour 11 nœuds équirépartis dans l'intervalle $[-1, 1]$.



Dans le cas particulier où les nœuds sont équirépartis on a le résultat suivant :

$$E_n(f) = \max_{x \in I} |f(x) - \Pi_n f(x)| \leq \frac{1}{4(n+1)} \left(\frac{b-a}{n} \right)^{n+1} \max_{x \in I} |f^{(n+1)}(x)|. \quad (3)$$

Démonstration. On peut montrer, comme on le voit d'ailleurs sur la figure précédente, que le maximum de $|\omega_{n+1}(x)|$ est atteint toujours dans un des deux intervalles extrêmes $[x_0, x_1]$ ou $[x_{n-1}, x_n]$. On prend alors $x \in [x_0, x_1]$ (l'autre cas est similaire); on a

$$|(x - x_0)(x - x_1)| \leq \frac{(x_1 - x_0)^2}{4} = \frac{h^2}{4}$$

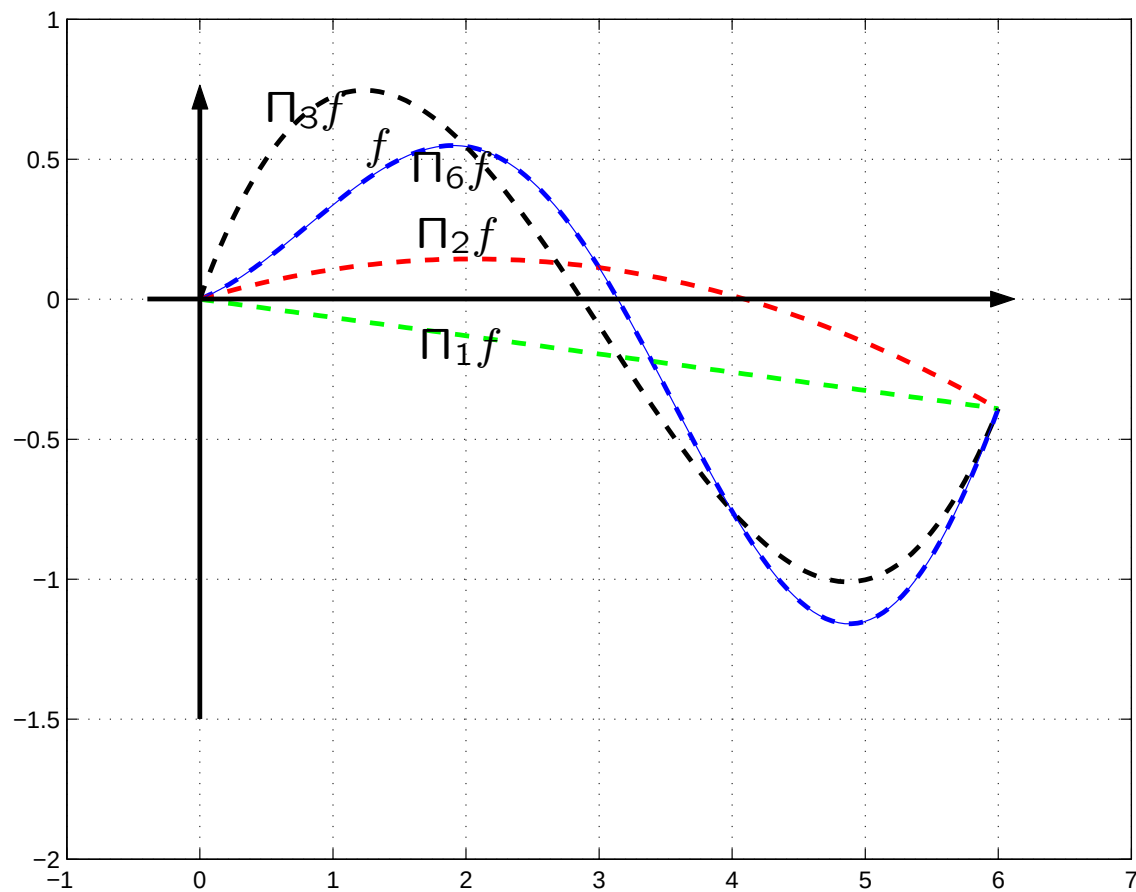
où l'on a noté $h = (b - a)/n$. De plus, pour tout $i > 1$ on a $|(x - x_i)| \leq ih$. Donc,

$$\max_{x \in I} \prod_{i=0}^n |(x - x_i)| \leq \frac{h^2}{4} 2h 3h \dots nh = \frac{h^{n+1} n!}{4} = \frac{n!}{4} \left(\frac{b-a}{n} \right)^{n+1}$$

d'où on obtient la relation (3).

On remarque que l'erreur d'interpolation dépend de la dérivée $n + 1$ -ième de f .

Exemple 4 Polynômes d'interpolation $\Pi_i f$ pour $i = 1, 2, 3, 6$ et $f(x) = \frac{x+1}{5} \sin x$, noeuds équirépartis sur $[0, 6]$.



Interpolation numérique en utilisant MATLAB

En MATLAB on peut calculer les polynômes d'interpolation en utilisant les commandes `polyfit` et `polyval`. Voyons plus en détail comment peut-on utiliser ces commandes.

`p = polyfit(x,y,n)` calcule les coefficients du polynôme de degré `n` qui interpole les valeurs `y` aux points `x`.

Exemple A. On veut interpoler les valeurs $y = [3.38, 3.86, 3.85, 3.59, 3.49]$ aux points $x = [0, 0.25, 0.5, 0.75, 1]$ par un polynôme de degré 4. Il suffit d'utiliser les commandes MATLAB suivantes:

```
>> x=[0:0.25:1]; % vecteur des points d'interpolation
>> y=[3.38 3.86 3.85 3.59 3.49]; % vecteur des valeurs
>> p1=polyfit(x,y,4)
p1 =
    1.8133   -0.1600   -4.5933    3.0500    3.3800
```

p1 est le vecteur des coefficients du polynôme interpolant: $\Pi_4(x) = 1.8133x^4 - 0.16x^3 - 4.5933x^2 + 3.05x + 3.38$.

Pour calculer le polynôme de degré n qui interpole une fonction f donnée dans $n + 1$ points, il faut d'abord construire le vecteur y en évaluant f dans les nœuds x .

Exemple B. Considérons le cas suivant:

```
>> f='cos(x)';  
>> x=[0:0.25:1];  
>> y=eval(f);  
>> p=polyfit(x,y,4)  
p =  
    0.0362    0.0063   -0.5025    0.0003    1.0000
```

REMARQUE Si la dimension $m + 1$ de x et y est $m + 1 > n + 1$ (n étant le degré du polynôme d'interpolation), la commande `polyfit(x,y,n)` retourne le polynôme interpolant de degré n au sens des moindres carrés (voir Sect. 8). Dans le cas où $m + 1 = n + 1$, alors on trouve le polynôme d'interpolation introduit dans la Sect. 3, puisque dans ce cas il coïncide avec celui aux moindres carrés.

$y = \text{polyval}(p,x)$ calcule les valeurs y d'un polynôme de degré n , dont les $n+1$ coefficients sont mémorisés dans le vecteur p , aux points x , c'est-à-dire: $y = p(1)*x.^n + \dots + p(n)*x + p(n+1)$.

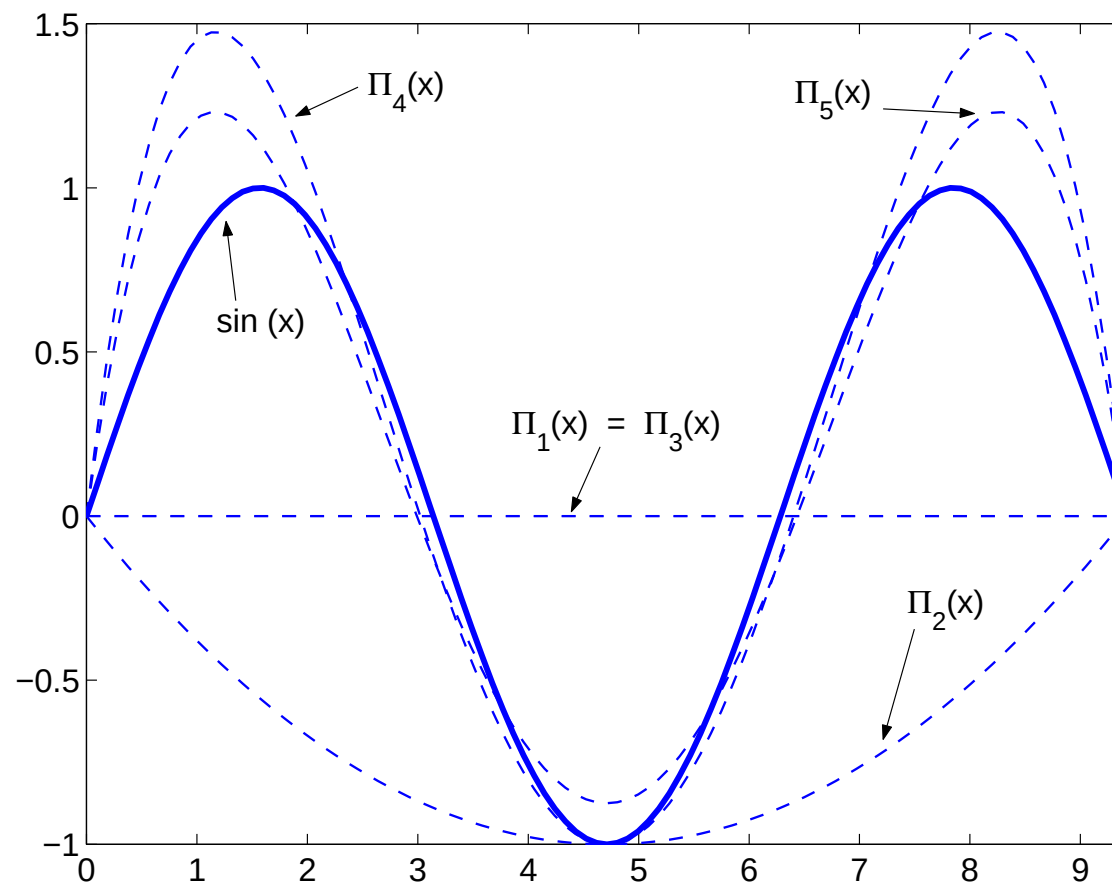
Exemple C. On veut évaluer le polynôme trouvé dans l'Exemple A dans le point $x = 0.4$ et, ensuite, on veut tracer son graphe. On peut utiliser les commandes suivantes:

```
>> x=0.4;  
>> y1=polyval(p1,x)  
y1 =  
    3.9012  
  
>> x=linspace(0,1,100);  
>> y=polyval(p1,x);  
>> plot(x,y)
```

Exemple 5 On veut interpoler la fonction $\sin(x)$ sur $n = 2, 3, 4, 5, 6$ nœuds. En Matlab on peut utiliser la commande `polyfit` pour calculer les coefficients du polynôme interpolant et `polyval` pour évaluer un polynôme dont on connaît les coefficients dans une suite de points. Voilà les commandes Matlab :

```
>> f = 'sin(x)';  
>> x=[0:3*pi/100:3*pi]; x_sample=x;  
>> plot(x,eval(f),'b'); hold on  
>> for i=2:6  
    x=linspace(0,3*pi,i); y=eval(f);  
    c=polyfit(x,y,i-1);  
    plot(x_sample,polyval(c,x_sample),'b--')  
end
```

La figure suivante montre les 5 polynômes obtenus.



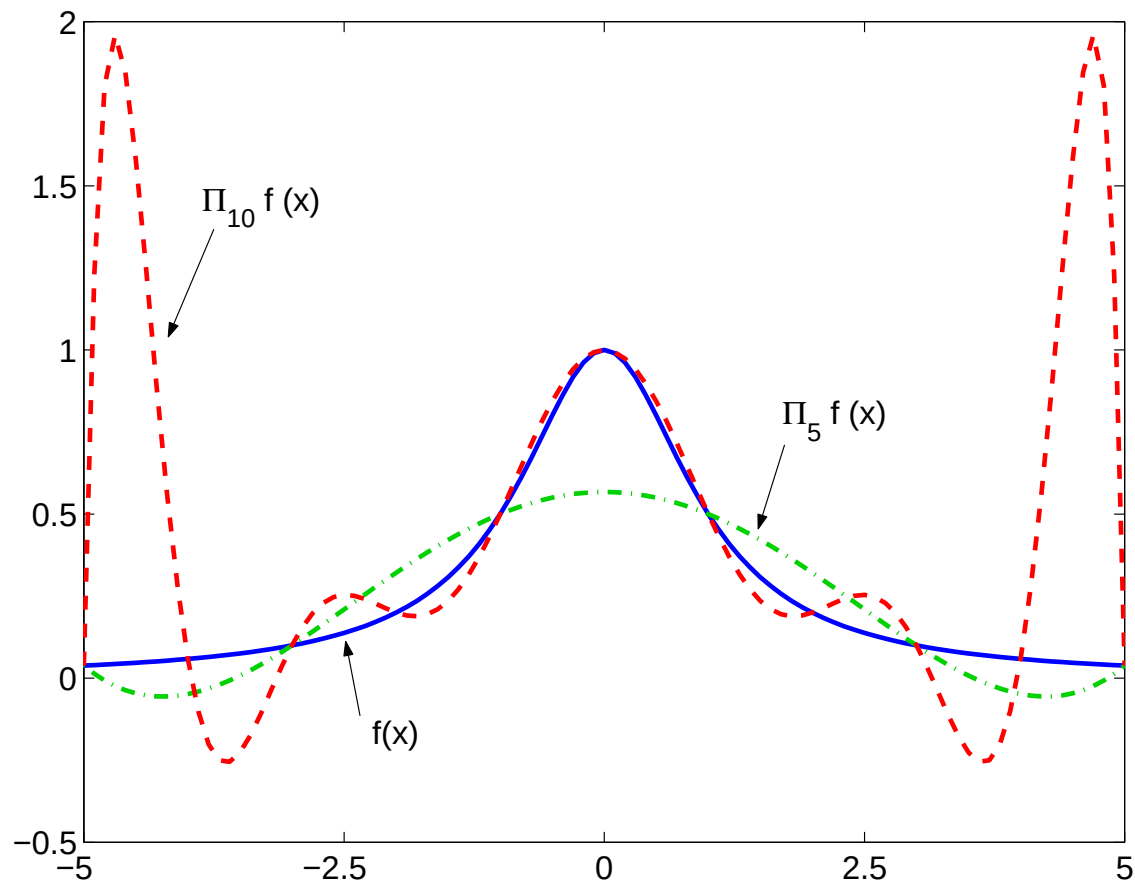
Remarque 1 *Seul le fait que*

$$\lim_{n \rightarrow \infty} \frac{1}{4(n+1)} \left(\frac{b-a}{n} \right)^{n+1} = 0$$

n'implique pas que $E_n(f)$ tend vers zéro quand $n \rightarrow \infty$.

Exemple 6 (Runge) Soit $f(x) = \frac{1}{1+x^2}$, $x \in [-5, 5]$. Si on l'interpole dans des points équirépartis, au voisinage des extrémités de l'intervalle, l'interpolant présente des oscillations, comme on peut le voir sur la figure.

Fonction de Runge et oscillations des polynômes interpolants dans des points équirépartis.



Remèdes :

1. Interpolation avec points qui ne sont pas équirépartis (interpolation de Chebyshev).
2. Interpolation par intervalles.

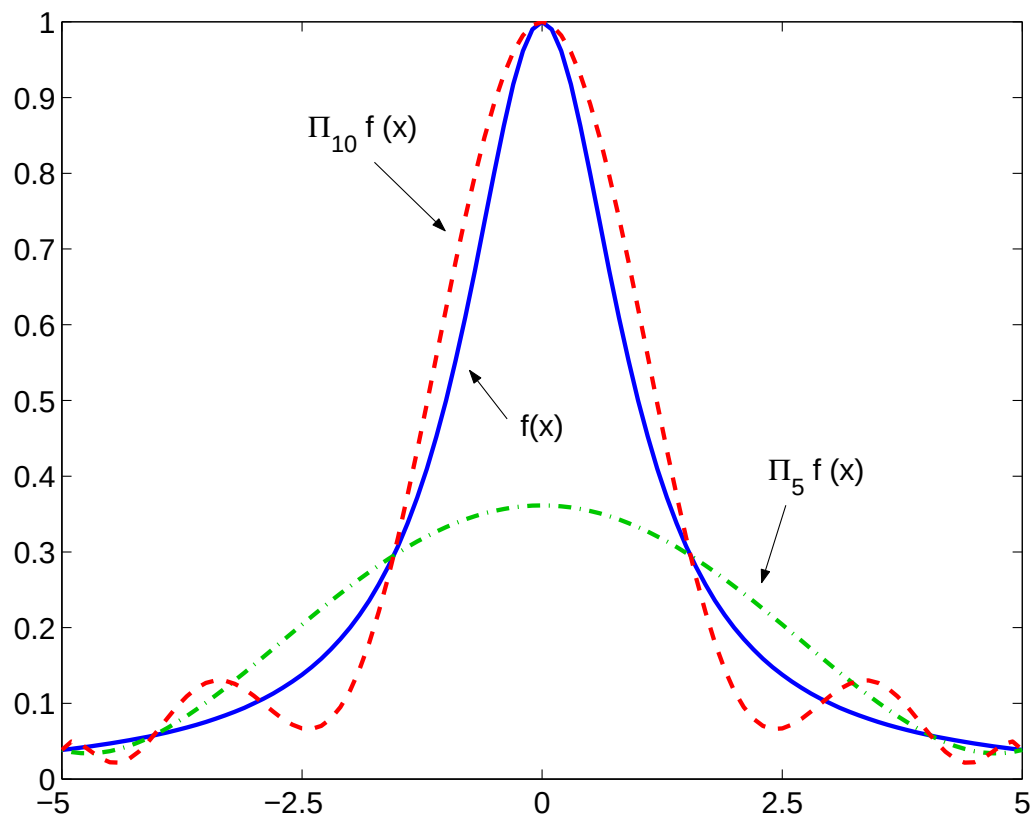
Interpolation de Chebyshev (Sec. 3.1.2 du livre)

Pour chaque entier positif $n \geq 1$, pour $i = 0, \dots, n$, on note $\hat{x}_i = -\cos(\pi i/n) \in [-1, 1]$ les *points de Chebyshev* et on définit

$$x_i = \frac{a+b}{2} + \frac{b-a}{2}\hat{x}_i \in [a, b],$$

pour un intervalle arbitraire $[a, b]$. Pour une fonction continue $f \in C^1([a, b])$, le polynôme d'interpolation $\Pi_n f$ de degré n aux noeuds $\{x_i, i = 0, \dots, n\}$ converge uniformément vers f quand $n \rightarrow \infty$.

Exemple 6 (suite) On reprend le même exemple mais on interpole la fonction de Runge dans les points de Chebyshev. La figure montre les polynômes de Chebyshev de degré $n = 5$ et $n = 10$. On remarque que les oscillations diminuent lorsque on augmente le degré du polynôme.



Interpolation par intervalles (Sec. 3.2 du livre)

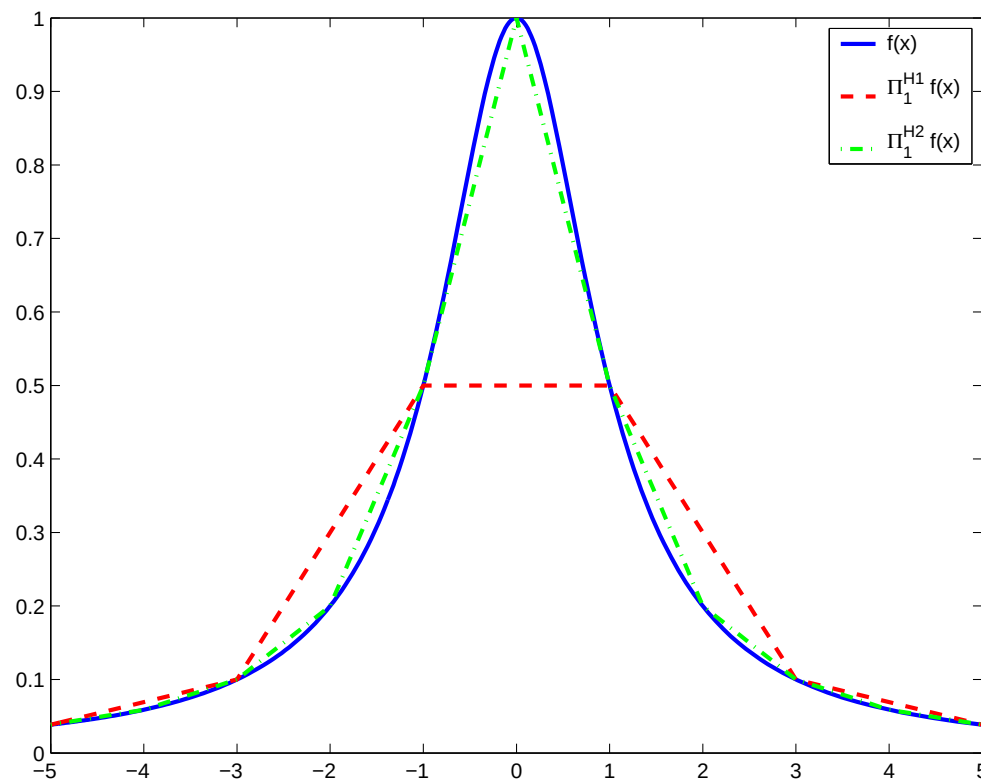
Soient $x_0 = a < x_1 < \cdots < x_N = b$ des points qui divisent l'intervalle $I = [a, b]$ dans une réunion d'intervalles $I_i = [x_i, x_{i+1}]$ de longueur H où

$$H = \frac{b - a}{N}.$$

Sur chaque sous-intervalle I_i on interpole $f|_{I_i}$ par un polynôme de degré 1. Le polynôme par morceaux qu'on obtient est noté $\Pi_1^H f(x)$, et on a:

$$\Pi_1^H f(x) = f(x_i) + \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}(x - x_i) \quad \text{pour } x \in I_i.$$

Exemple 6 (suite) On considère les polynômes par morceaux de degré $n = 1$ interpolants la fonction de Runge, pour 5 et 10 sous-intervalles de $[-5, 5]$.



La figure montre les polynômes $\Pi_1^{H_1} f$ et $\Pi_1^{H_2} f$ pour $H_1 = 2.0$ et $H_2 = 1.0$ obtenus par Matlab.

Les commandes Matlab utilisées sont:

```
>> f = inline('1./(1+x.^2)','x');  
>> H1 = 2.0; H1=1.0;  
>> x1 = [-5:H1:5]; x2 = [-5:H2:5];  
>> y1 = feval(f, x1); y2 = feval(f, x2);  
>> x_plot = [-5:.1:5];  
>> y1_plot = interp1(x1,y1,x_plot);  
>> y2_plot = interp1(x1,y1,x_plot);  
>> fplot(f, [-5 5], 'b'); hold on  
>> plot(x_plot, y1_plot, '--g'); plot(x_plot, y2_plot, '_.r')
```

Théorème 2 Si $f \in C^2(I)$, ($I = [a, b]$) alors il existe une constante $C > 0$ telle que

$$E_1^H(f) = \max_{x \in I} |f(x) - \Pi_1^H f(x)| \leq \frac{H^2}{8} \max_{x \in I} |f''(x)|.$$

Démonstration. D'après la formule (3), sur chaque intervalle I_i on a

$$\max_{x \in [x_i, x_{i+1}]} |f(x) - \Pi_1^H f(x)| \leq \frac{H^2}{4(1+1)} \max_{x \in I_i} |f''(x)|.$$

Remarque 2 On peut montrer que si l'on utilise un polynôme de degré $n (\geq 1)$ dans chaque sous-intervalle I_i on trouve

$$E_n^H(f) \leq \frac{H^{n+1}}{4(n+1)} \max_{x \in I} |f^{(n+1)}(x)|.$$

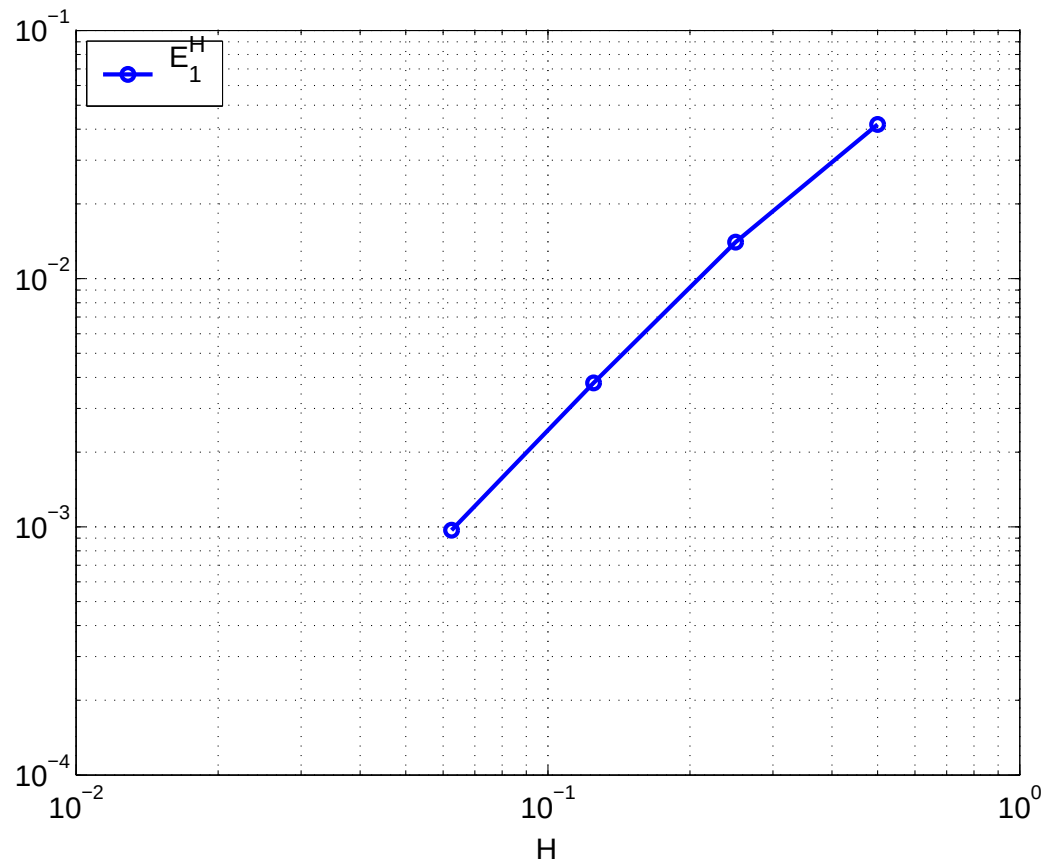
Exemple 6 (suite) On considère la fonction de Runge $f(x)$ sur $[-5, 5]$ et on prend un nombre K croissant de sous-intervalles $K = 20, 40, 80, 160$ et on estime l'erreur d'interpolation commise en évaluant la différence $|f(x) - \Pi_1^H f(x)|$ sur une grille très fine :

```
>> K=[20 40 80 160]; H=10./K;
>> x_fine = [-5:0.001:5];
>> f_fine = feval(f,x_fine);
>> for i=1:4
    x = [-5:H(i):5]; y = feval(f, x);
    y_fine = interp1(x,y,x_fine);
    err1(i) = max(abs(f_fine - y_fine));
end
>> loglog(H,err1,'-bo')
```

La figure qui suit montre (en échelle logarithmique) l'erreur commise en fonction de la taille $H = 10/K$ des sous-intervalles. On voit que l'erreur $E_1^H f$ pour l'interpolation linéaire par morceau se comporte comme CH^2 : ce résultat est en accord avec le théorème (2). En plus, si on calcule les rapports E_1^H/H^2 on peut estimer les constantes C :

```
>> err1./H.^2
ans =
    0.1673    0.2247    0.2433    0.2483
```

Erreur d'interpolation de la fonction de Runge par le polynôme composite Π_1^H en fonction de H .



Interpolation par fonctions splines (Sec. 3.3 du livre)

Soient $a = x_0 < x_1 < \dots < x_n = b$ des points qui divisent l'intervalle $I = [a, b]$ dans une réunion d'intervalles $I_i = [x_i, x_{i+1}]$.

Définition 1 On appelle *spline cubique interpolant* f une fonction s_3 qui satisfait

1. $s_3|_{I_i} \in \mathbb{P}_3$ pour tout $i = 0, \dots, n-1$, $\mathbb{P}_3$ étant l'ensemble de polynômes de degré 3,
2. $s_3(x_i) = f(x_i)$ pour tout $i = 0, \dots, n$,
3. $s_3 \in C^2([a, b])$.

Cela revient à vérifier les conditions suivantes (on indique par $s_3(x_i^-)$ la limite à gauche de s_3 au point x_i et par $s_3(x_i^+)$ la limite à droite) :

$$\begin{array}{ll}
 s_3(x_i^-) = f(x_i) & \text{pour tout } 1 \leq i \leq n-1, \\
 s_3(x_i^+) = f(x_i) & \text{pour tout } 1 \leq i \leq n-1, \\
 s_3(x_0) = f(x_0), & \\
 s_3(x_n) = f(x_n), & \\
 s'_3(x_i^-) = s'_3(x_i^+) & \text{pour tout } 1 \leq i \leq n-1, \\
 s''_3(x_i^-) = s''_3(x_i^+) & \text{pour tout } 1 \leq i \leq n-1,
 \end{array}$$

c'est-à-dire $2(n-1) + 2 + 2(n-1) = 4n - 2$ conditions.

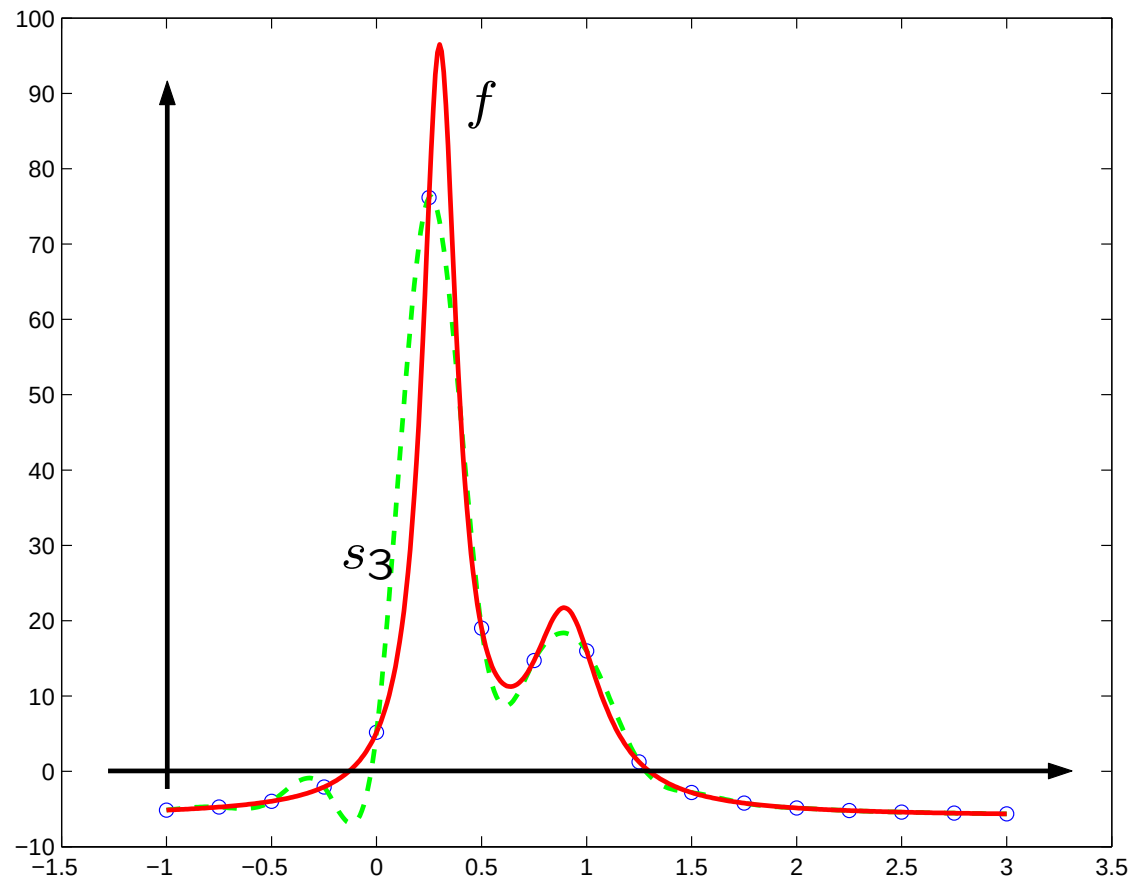
Il faut trouver $4n$ inconnues (qui sont les 4 coefficients de chacune des n restrictions $s_3|_{I_i}$, $i = 0, \dots, n-1$) et on dispose de $4n-2$ relations.

On rajoute alors 2 conditions supplémentaires à vérifier :

$$s_3''(x_0^+) = 0 \quad \text{et} \quad s_3''(x_n^-) = 0. \quad (4)$$

Avec ces conditions la spline s_3 est complètement déterminée et s'appelle *spline naturelle*.

Spline cubique naturelle interpolant la fonction $f(x) = \frac{1}{(x-0.3)^2+0.01} + \frac{1}{(x-0.9)^2+0.04} - 6$,
 noeuds équirépartis $x_j = -1 + j/4, j = 0, \dots, 16$.



Approximation au sens des moindres carrés (Sec. 3.4 du livre)

On suppose de disposer de $n+1$ points $x_0, x_1, \dots, x_n$ et $n+1$ valeurs $y_0, y_1, \dots, y_n$. On a vu que, si le nombre de données est grand, le polynôme interpolant peut présenter des oscillations importantes.

Pour avoir une meilleure représentation des données on peut chercher un polynôme de degré $m < n$ qui approche “au mieux” les données.

Définition 2 On appelle polynôme aux moindres carrés $\tilde{f}_m(x)$ le polynôme de degré m tel que

$$\sum_{i=0}^n |y_i - \tilde{f}_m(x_i)|^2 \leq \sum_{i=0}^n |y_i - p_m(x_i)|^2 \quad \forall p_m(x) \in \mathbb{P}_m$$

Remarque 3 Lorsque $y_i = f(x_i)$ (f étant une fonction continue) alors $\tilde{f}_m$ est dit l'approximation de f au sens des moindres carrés.

Autrement dit, le polynôme aux moindres carrés est le polynôme de degré m qui, parmi tous les polynômes de degré m , minimise la distance des données.

Si on note $\tilde{f}_m(x) = a_0 + a_1x + a_2x^2 + \dots + a_mx^m$, et on définit la fonction

$$\Phi(a_0, a_1, \dots, a_m) = \sum_{i=0}^n \left| y_i - (a_0 + a_1x_i + a_2x_i^2 + \dots + a_mx_i^m) \right|^2$$

alors les coefficients du polynôme aux moindres carrés peuvent être déterminés par les relations

$$\frac{\partial \Phi}{\partial a_k} = 0, \quad 0 \leq k \leq m \tag{5}$$

ce qui nous donne $m + 1$ relations linéaires entre les a_k .

En effet on trouve:

$$\begin{aligned}\frac{\partial \Phi}{\partial a_0} &= -2 \sum_{i=0}^n [y_i - (a_0 + \dots + a_m x_i^m)] \\ \frac{\partial \Phi}{\partial a_1} &= -2 \sum_{i=0}^n x_i [y_i - (a_0 + \dots + a_m x_i^m)],\end{aligned}$$

et en général on peut écrire

$$\begin{aligned}\frac{\partial \Phi}{\partial a_k} &= -2 \sum_{i=0}^n x_i^k [y_i - (a_0 + \dots + a_m x_i^m)] \\ &= -2 \left[\sum_{i=0}^n x_i^k y_i - \left(a_0 \sum_{i=0}^n x_i^k + a_1 \sum_{i=0}^n x_i^{k+1} + \dots + a_m \sum_{i=0}^n x_i^{k+m} \right) \right]\end{aligned}$$

pour tout $k = 0, \dots, m$.

Alors, en imposant les (5) on obtient le système linéaire suivant dans les inconnues

$a_0, \dots, a_m$:

$$\left\{ \begin{array}{l} a_0(n+1) + a_1 \sum_{i=0}^n x_i + \dots + a_m \sum_{i=0}^n x_i^m = \sum_{i=0}^n y_i \\ a_0 \sum_{i=0}^n x_i + a_1 \sum_{i=0}^n x_i^2 + \dots + a_m \sum_{i=0}^n x_i^{m+1} = \sum_{i=0}^n y_i x_i \\ \vdots \\ a_0 \sum_{i=0}^n x_i^m + a_1 \sum_{i=0}^n x_i^{m+1} + \dots + a_m \sum_{i=0}^n x_i^{2m} = \sum_{i=0}^n y_i x_i^m \end{array} \right.$$

Le système linéaire qu'on a obtenu peut être réécrit comme $Aa = y$, où

$$A = \begin{pmatrix} n+1 & \cdots & \sum_{i=0}^n x_i^m \\ \vdots & & \vdots \\ \sum_{i=0}^n x_i^m & \cdots & \sum_{i=0}^n x_i^{2m} \end{pmatrix} \quad \text{et} \quad y = \begin{pmatrix} \sum_{i=0}^n y_i \\ \vdots \\ \sum_{i=0}^n y_i x_i^m \end{pmatrix} \quad (6)$$

et $a = (a_0, \dots, a_m)^T$ est le vecteur des inconnues.

En particulier, si $m = 1$, on obtient la *ligne de regression* $\tilde{f}_1(x) = a_0 + a_1x$ où $\{a_0, a_1\}$ sont la solution du système linéaire:

$$\begin{pmatrix} n+1 & \sum_{i=0}^n x_i \\ \sum_{i=0}^n x_i & \sum_{i=0}^n x_i^2 \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \end{pmatrix} = \begin{pmatrix} \sum_{i=0}^n y_i \\ \sum_{i=0}^n y_i x_i \end{pmatrix} \quad (7)$$

Utilisons cette méthode dans un cas simple. Considérons les points $x_0 = 1$, $x_1 = 3$, $x_2 = 4$ et les valeurs $y_0 = 0$, $y_1 = 2$, $y_2 = 7$ et calculons le polynôme interpolant de degré 1 au sens des moindres carrés (*ligne de regression*).

Le polynôme cherché a la forme $\tilde{f}_1(x) = a_0 + a_1x$. On définit: $\Phi(a_0, a_1) = \sum_{i=0}^2 [y_i - (a_0 + a_1x_i)]^2$ et on impose $\frac{\partial \Phi}{\partial a_0} = 0$ et $\frac{\partial \Phi}{\partial a_1} = 0$:

$$\begin{aligned}\frac{\partial \Phi}{\partial a_0} &= -2 \sum_{i=0}^2 [y_i - (a_0 + a_1x_i)] = -2 \left(\sum_{i=0}^2 y_i - 3a_0 - a_1 \sum_{i=0}^2 x_i \right) \\ &= -2(9 - 3a_0 - 8a_1) \\ \frac{\partial \Phi}{\partial a_1} &= -2 \sum_{i=0}^2 x_i [y_i - (a_0 + a_1x_i)] = -2 \left(\sum_{i=0}^2 x_i y_i - a_0 \sum_{i=0}^2 x_i - a_1 \sum_{i=0}^2 x_i^2 \right) \\ &= -2(34 - 8a_0 - 26a_1)\end{aligned}$$

Donc les coefficients a_0 et a_1 du polynôme sont les solutions du système linéaire:

$$\begin{cases} 3a_0 + 8a_1 = 9 \\ 8a_0 + 26a_1 = 34 \end{cases}$$

qui correspond à (7) pour $n = 2$.

On observe que si pour calculer le polynôme interpolant aux moindres carrés $\tilde{f}_m(x) = a_0 + a_1x + a_2x^2 + \dots + a_mx^m$ on impose les conditions d'interpolation $\tilde{f}_m(x_i) = y_i$ pour $i = 0, \dots, n$, alors on trouve le système linéaire $B\mathbf{a} = \tilde{\mathbf{y}}$, où B est la matrice de dimension $(n + 1) \times (m + 1)$

$$B = \begin{pmatrix} 1 & x_1 & \dots & x_1^m \\ 1 & x_2 & \dots & x_2^m \\ \vdots & & & \vdots \\ 1 & x_n & \dots & x_n^m \end{pmatrix}$$

Puisque $m < n$ le système est surdéterminé, c'est-à-dire que le nombre de lignes est plus grand que le nombre de colonnes. Donc on ne peut pas résoudre ce système de façon classique, mais on doit le résoudre au sens des moindres carrés, ce qui équivaut à considérer le système

$$B^T B \mathbf{a} = B^T \tilde{\mathbf{y}}$$

De cette façon on trouve le système linéaire $A\mathbf{a} = \mathbf{y}$, dit *système d'équations normales*, avec A et $\mathbf{y}$ définis par (6).

Exemple 2 (suite) On reprend l'exemple 2 proposé au début du chapitre. On cherche à deviner la valeur de l'action au début du mois de Juillet 2000, donc pour en temps $t > 05/06/2000$. Une technique possible consiste à calculer une fonction approchant les données et à regarder le comportement de cette fonction en dehors de l'intervalle d'interpolation pour $t > 05/06/2000$. Dans la suite, on va considérer seulement les valeurs de l'action au début de chaque mois :

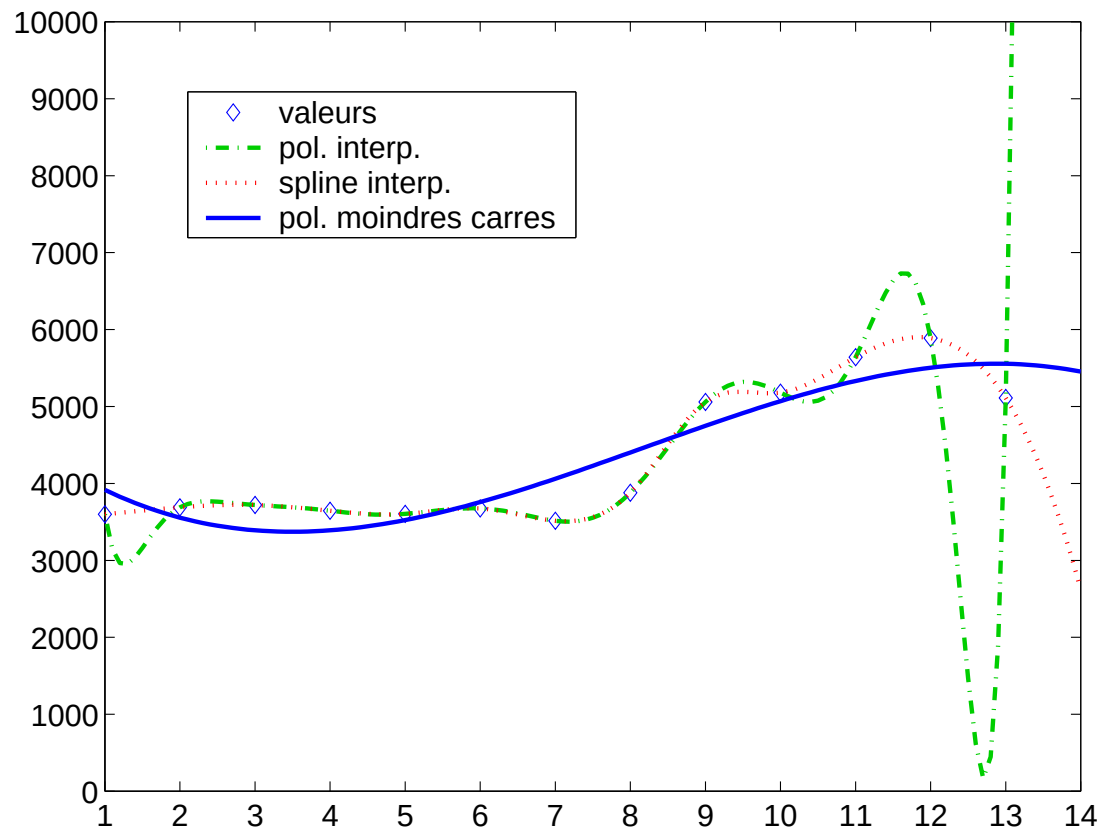
```
>> val = [3600, 3690, 3720, 3645, 3605, 3675, 3515, 3880, 5060, 5180,
5640, 5890, 5110];
```

et on considère comme fonctions approchantes, le polynôme et la spline interpolant les données ainsi que le polynôme aux moindres carrés de degré 3.

On va obtenir les trois graphes des fonctions considérées. On peut utiliser les commandes suivants:

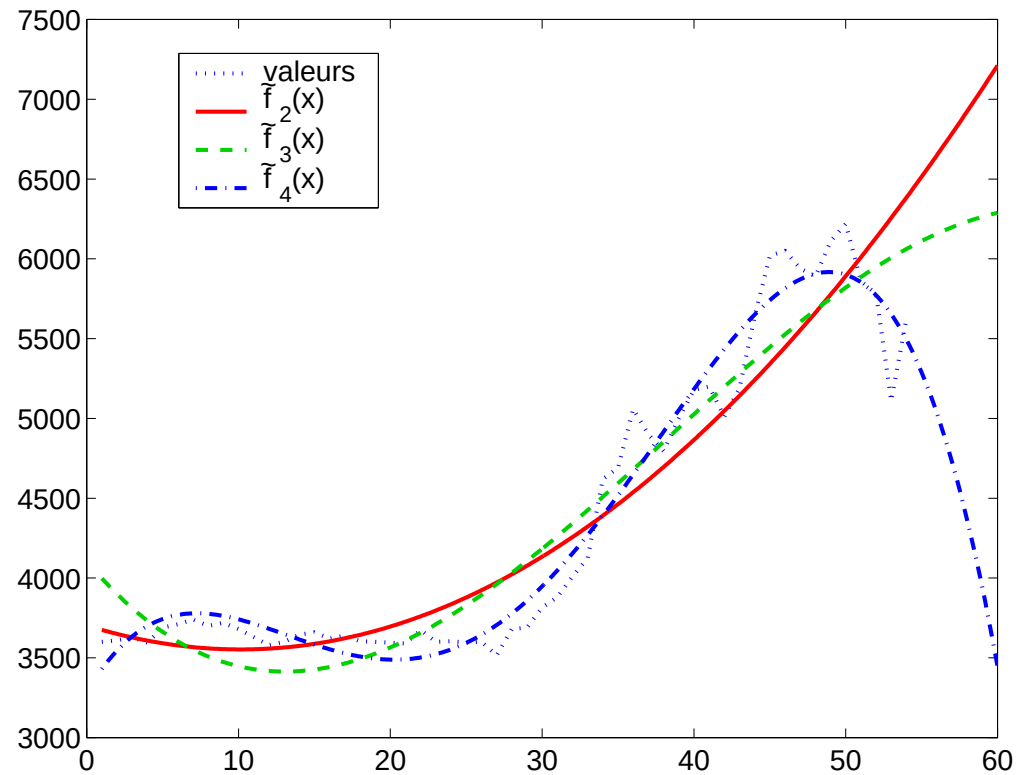
```
>> x=[1:1:13]; x_sample = [1:0.1:14];
>> c=polyfit(x,val,12); pol = polyval(c,x_sample); % interpolation polynomial
>> plot(x,val,'db',x_sample,pol,'--b'); hold on
>> s=spline(x,val,x_sample); % spline
>> plot(x_sample,s,':b')
>> c1=polyfit(x,val,3); pol_mc = polyval(c1,x_sample); % moindres carres
>> plot(x_sample,pol_mc,'b-.'); hold off
```

La figure montre le polynôme interpolant, la spline interpolant et le polynôme aux moindres carrés de degré 3.



On observe le phénomène typique d'oscillation du polynôme interpolant.

Finalement, la figure ci-dessous montre les polynômes aux moindres carrés de degré 2, 3 et 4 obtenus à partir de toutes les valeurs disponibles.



Comme on le voit, le choix correct du degré du polynôme aux moindres carrés (sur la base du comportement des données) est cruciale pour obtenir des prévisions raisonnables.

Applications

On revient à l'exemple proposé au début du chapitre.

Exemple 1 (suite) On considère la colonne de valeurs relative à la concentration $K = 0.67$. Pour interpoler les valeurs données on peut utiliser une interpolation polynômiale.

```
>> lat = [65:-10:-55]; x_sample = [-55:1:65];  
>> d_temp = [-3.106 -3.228 -3.309 -3.327 -3.172 -3.074 -3.029 -3.029 ...  
             -3.124 -3.209 -3.350 -3.373 -3.258];  
>> n = size(lat,2) - 1  
>> c = polyfit(lat,d_temp,n);  
>> pol = polyval(c, x_sample);  
>> plot(lat,d_temp,'o',x_sample,pol,'b'); hold on
```

La variation de température estimée pour la latitude de Lausanne est donc

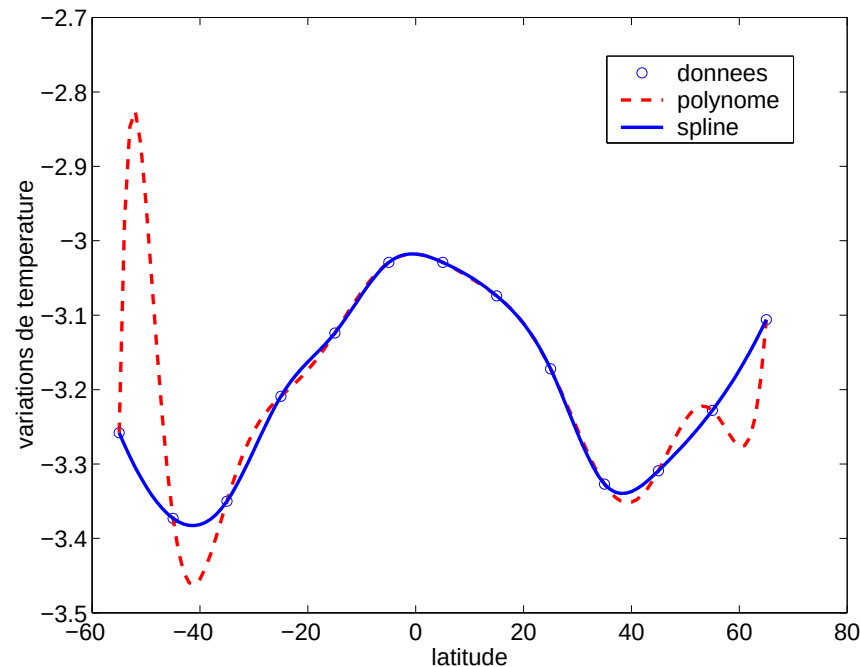
```
>> polyval(c, 46.316)  
ans =  
    -3.2904
```

Toutefois, comme on l'a vu dans le cours, l'interpolation polynômiale sur des nœuds équirépartis présente des oscillations au voisinage des valeurs extrêmes. La valeur estimée pour la latitude $l = 46.316$ n'est pas trop fiable.

On calcule, maintenant, la spline interpolant en utilisant la commande `spline` de Matlab (à *savoir* : l'algorithme de Matlab n'utilise pas les deux conditions supplémentaires (4) mais il impose la continuité de la dérivée troisième aux nœuds x_1 et x_{n-1} (*not-a-knot* condition)).

```
>> s = spline(lat,d_temp,x_sample);  
>> plot(x_sample,s)
```

La figure montre le polynôme et la spline interpolants. On voit bien que la spline est beaucoup plus lisse (c-à-d ne présente pas d'oscillation) que le polynôme.



Appendice: Traitement des polynômes dans MATLAB

Dans MATLAB il y a des commandes spécifiques pour faire des calculs avec des polynômes. Soit x un vecteur de abscisses, y un vecteur d'ordonnées et p (respectivement p_i) le vecteur des coefficients d'un polynôme $P(x)$ (respectivement P_i); alors on a les commandes suivantes:

commande	action
<code>y=polyval(p,x)</code>	$y = \text{valeurs de } P(x)$
<code>p=polyfit(x,y,n)</code>	$p = \text{coefficients du polynôme d'interpolation } \Pi_n$
<code>z=roots(p)</code>	$z = \text{zéros de } P \text{ tels que } P(z) = 0$
<code>p=conv(p1,p2)</code>	$p = \text{coefficients du polynôme } P_1 P_2$
<code>[q,r]=deconv(p1,p2)</code>	$q = \text{coefficients de } Q, r = \text{coefficients de } R$ tels que $P_1 = QP_2 + R$
<code>y=polyder(p)</code>	$y = \text{coefficients de } P'(x)$
<code>y=polyint(p)</code>	$y = \text{coefficients de } \int P(x) dx$